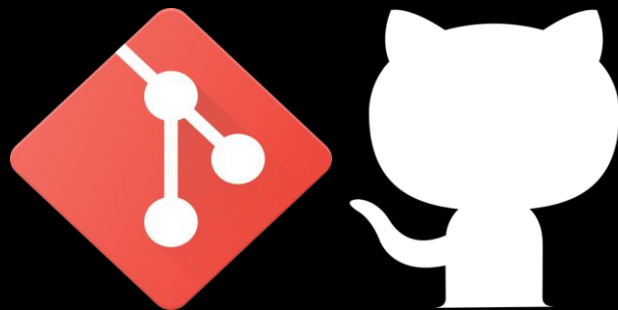


GIT & GITHUB



WHAT WE WILL DISCUSS ?

- What is Git ?
- What is GitHub?
- Git vs GitHub
- Git Workflow
- Setting up Github Account
- Setting up Git Account
- Git Commands
- Github Repository
- Git and Github Task for Session

GIT

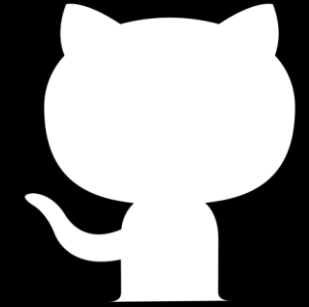


Version Control System is a tool that helps to track changes in code

Git is a **Version Control System** . It is :

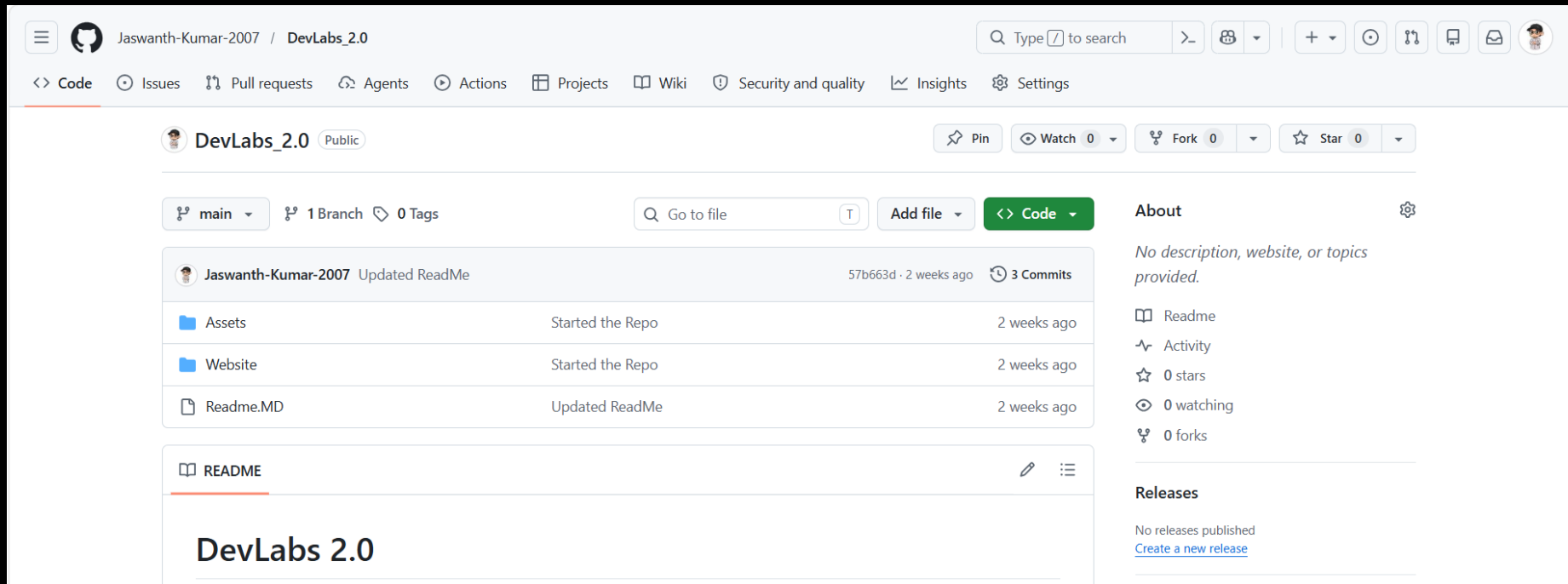
- Popular
- Free & Open Source
- Fast and Scalable
- Saving project history
- Tracking changes
- Collaboration

GITHUB



Website that Allows Developers to store and Manage their code using Git

<https://github.com>



The screenshot shows the GitHub interface for a repository named 'DevLabs_2.0' by user 'Jaswanth-Kumar-2007'. The repository is public and has 0 stars, 0 forks, and 0 watchers. The main branch is 'main', and there is 1 branch and 0 tags. The repository contains three files: 'Assets', 'Website', and 'Readme.MD'. The 'Readme.MD' file is selected, showing the title 'DevLabs 2.0'. The right sidebar shows the 'About' section with no description, website, or topics provided, and the 'Releases' section with no releases published.

Navigation bar: <> Code, Issues, Pull requests, Agents, Actions, Projects, Wiki, Security and quality, Insights, Settings

Repository: DevLabs_2.0 (Public)

Buttons: Pin, Watch 0, Fork 0, Star 0

Branches: main (1 Branch), 0 Tags

Search: Go to file

Buttons: Add file, <> Code

Commit history:

Commit	Author	Message	Time
57b663d	Jaswanth-Kumar-2007	Updated ReadMe	2 weeks ago
	Jaswanth-Kumar-2007	Started the Repo	2 weeks ago
	Jaswanth-Kumar-2007	Started the Repo	2 weeks ago
	Jaswanth-Kumar-2007	Updated ReadMe	2 weeks ago

Files:

- Assets: Started the Repo (2 weeks ago)
- Website: Started the Repo (2 weeks ago)
- Readme.MD: Updated ReadMe (2 weeks ago)

Readme:

DevLabs 2.0

About:

No description, website, or topics provided.

- Readme
- Activity
- 0 stars
- 0 watching
- 0 forks

Releases:

No releases published

[Create a new release](#)

DEVLABS 2.0

Why Developers Use GitHub ?

1. Collaboration

Multiple developers can work on the same project together.

- Share code
- Review code
- Work in teams

2. Backup and Cloud Storage

Projects are stored online safely.

Benefits:

- Prevents data loss
- Access project from anywhere

3. Version Control

GitHub stores project history.

Developers can:

- Track changes
- Restore old versions
- Compare updates

4. Open Source Contributions

Developers contribute to public projects.

Examples:

- Fix bugs
- Add features
- Learn from real projects

5. Portfolio for Developers

GitHub acts like a coding portfolio.

Developers showcase:

- Projects
- Skills
- Contributions

6. Team Management

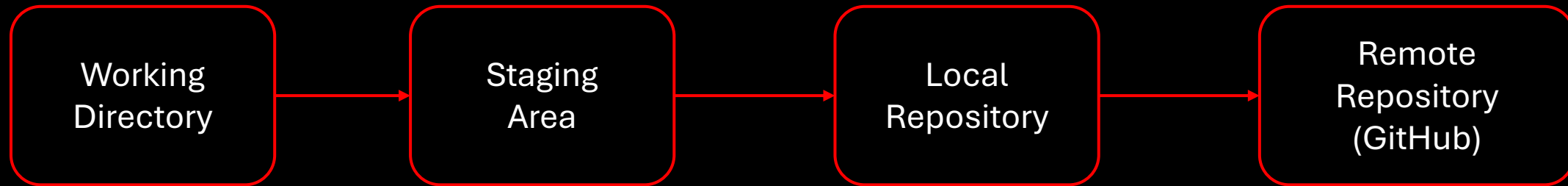
GitHub helps teams manage projects using:

- Pull Requests
- Issues
- Branches
- Code Reviews

GIT Vs GITHUB

Git	GitHub
Works in a decentralized architecture where every developer has a full copy of the repository.	Works on a centralized hosting model for sharing and managing repositories online.
Operates mainly via command-line interface (CLI) with optional GUI tools.	Provides a web-based UI + integrations along with Git command support.
Installed and runs on the local machine.	Hosted on the cloud (web platform).
Focuses on version control operations like commit, branch, merge and rebase.	Focuses on collaboration workflows like pull requests, issues and code reviews.
Maintains complete history and snapshots of the project locally.	Hosts repositories and manages team collaboration and access control.
Open-source and maintained by the community (originally by Linus Torvalds).	Owned and maintained by Microsoft.
Does not include built-in access control or user management.	Provides role-based access control and permissions.
Competes with SVN, Mercurial, CVS.	Competes with GitLab, Bitbucket, Azure DevOps.

GIT Workflow

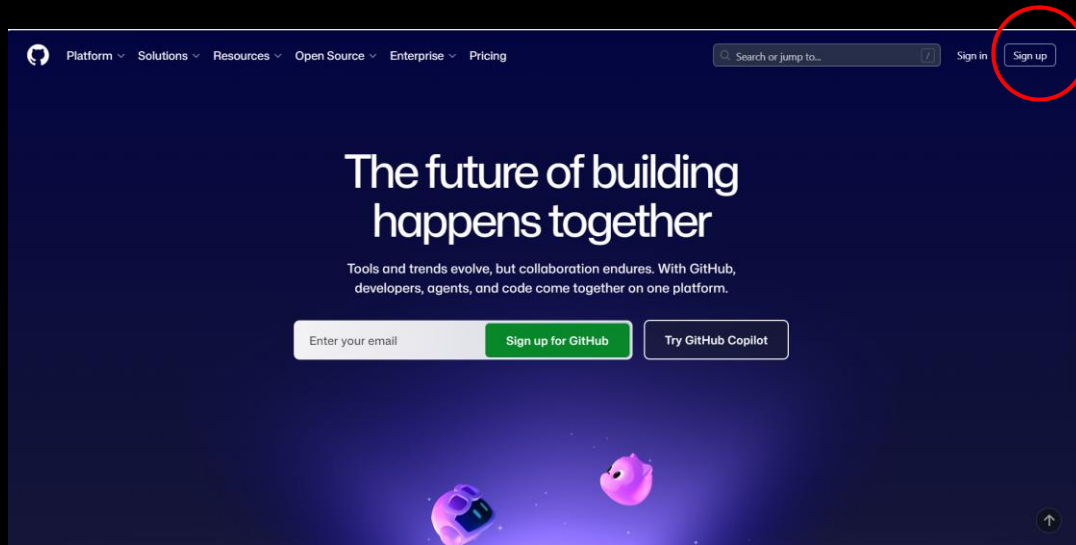
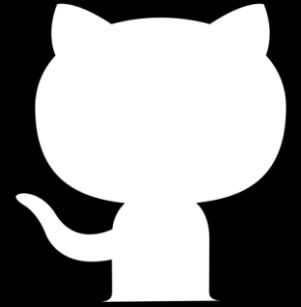


- Working Directory - Where you make changes
- git add - Stage changes
- git commit - Save changes to your repository
- git push - Share changes with others
- git status - Check what's going on
- Undo/Amend - Fix mistakes (`git restore`, `git reset`, `git commit --amend`)

Workflow Diagram

```
[Working Directory] --git add--> [Staging Area] --git commit--> [Repository]
```

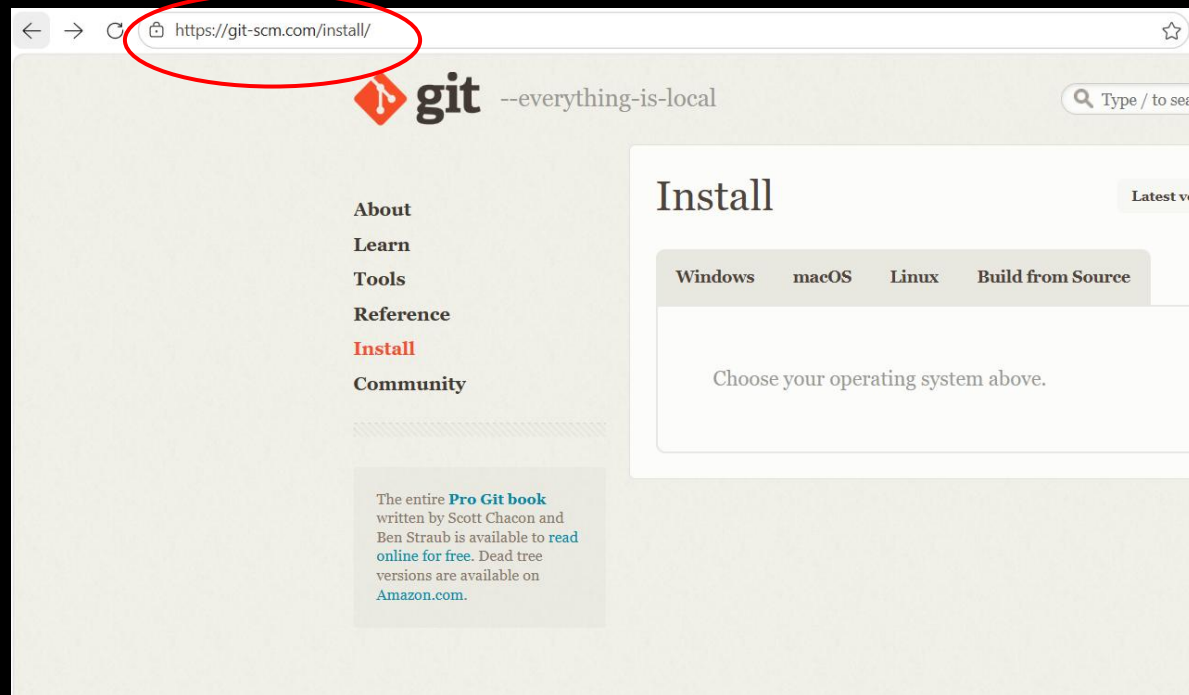
SETTING UP GITHUB ACCOUNT

A screenshot of the GitHub account creation page. The title is 'Create your free account'. Below it, there are links for 'Continue with Google' and 'Continue with Apple'. The form includes fields for 'Email', 'Password', and 'Username'. There is also a 'Your Country/Region' dropdown menu set to 'India'. At the bottom, there is a checkbox for 'Email preferences' and a 'Create account' button.

SETTING UP GIT



<https://git-scm.com/install/>



SETTING UP GIT



Check Git is Successfully Installed on your Device

Visual Studio Code

Windows (Git Bash)

Mac (Terminal)

`git --version`

```
Microsoft Windows [Version 10.0.26200.8328]  
(c) Microsoft Corporation. All rights reserved.
```

```
C:\Users\kamir>git --version  
git version 2.52.0.windows.1
```

CONFIGURING GIT



Next , Open VS Code Terminal

```
git config --global user.name "Your Name"
```

```
git config --global user.email youremail@gmail.com
```

```
git config --list
```

Git uses your name and email to label your commits.

GIT COMMANDS



git init : Initialize a new Git Repository in the current directory

git init

What Happens When You Run **git init** ?

- Git creates a hidden folder called .git inside your project.
- This is where Git stores all the information it needs to track your files and history.

git status : Display the state of the working directory & staging area

git status

GIT COMMANDS



What is an Untracked file

- An **untracked file** is any file in your project folder that Git is not yet tracking.
- These are files you've created or copied into the folder, but haven't told Git to watch.

What is a Tracked file

- A **tracked file** is a file that Git is watching for changes.
- To make a file tracked, you need to add it to the staging area

GIT COMMANDS



Staging Environment

- `git add <file>` - Stage a file
- `git add --all` or `git add -A` - Stage all changes
- `git status` - See what is staged
- `git restore --staged <file>` - Unstage a file

git add : Add Changes to the staging area

How to unstage a File

`git restore --staged index.html`

`git reset HEAD index.html`

`git add file.txt`

GIT COMMANDS



git commit : Record Changes to the Repository

```
git commit -m "Initial Commit"
```

- `git commit -m "message"` - Commit staged changes with a message
- `git commit -a -m "message"` - Commit all tracked changes (skip staging)
- `git log` - See commit history

Commit all changes with staging (-a)

```
git commit -a -m "message"
```

Note: `git commit -a` does not work for new/untracked files. You must use `git add <file>` first for new files.

GIT COMMANDS



Other Useful Commit Options

- Create an empty commit:

```
git commit --allow-empty -m "Start project"
```

- Use previous commit message (no editor):

```
git commit --no-edit
```

- Quickly add staged changes to last commit, keep message:

```
git commit --amend --no-edit
```

git log : Shows the Commit History

- `git log` - Show full commit history
- `git log --oneline` - Show a summary of commits
- `git show <commit>` - Show details of a specific commit
- `git diff` - See unstaged changes
- `git diff --staged` - See staged changes

GIT COMMANDS



git diff : different between your working directory and the last commit (unstaged changes)

git diff --staged : different between your staged files and the last commit

git diff <commit1> <commit2> : git diff 1234567 89abcde

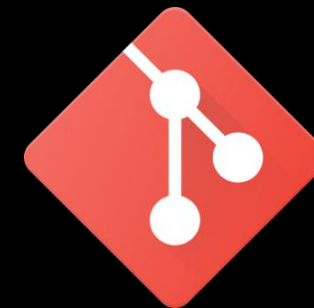
git log --author="Alice" :

```
git log --author="Alice"  
commit 1a2b3c4d5e6f7g8h9i0j  
Author: Alice  
Date:   Mon Mar 22 10:12:34 2021 +0100
```

Add about page

GIT COMMANDS

Undo a Commit



Step 1: Initial Code

app.js

```
</> JavaScript
console.log("Hello");
```



Commit it:

```
</> Bash
git add .
git commit -m "Initial commit"
```



Step 2: Add Dark Mode

Now change code:

```
</> JavaScript
console.log("Hello");

console.log("Dark mode added");
```



Commit again:

```
</> Bash
git add .
git commit -m "Added dark mode"
```



GIT COMMANDS



Undo a Commit

Current History

Commit 1 → Initial commit
Commit 2 → Added dark mode



Current file:

```
</> JavaScript  
  
console.log("Hello");  
  
console.log("Dark mode added");
```



Step 3: Revert Commit 2

Check log:

```
</> Bash  
  
git log
```



Suppose commit id is:

```
abc123 Added dark mode
```

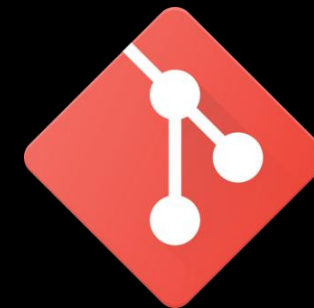


Now run:

```
</> Bash  
  
git revert abc123
```



GIT COMMANDS



Undo a Commit

Git creates NEW commit:

```
Commit 1 → Initial commit  
Commit 2 → Added dark mode  
Commit 3 → Revert "Added dark mode"
```



Final Code After Revert

```
</> JavaScript  
  
console.log("Hello");
```



Can Use Git Reset

```
git reset --hard
```

After reset:

```
Commit 1 → Initial commit
```



Code:

```
</> JavaScript  
  
console.log("Hello");
```



Commit 2 completely disappears.

GIT COMMANDS



git init : Initialize a new Git Repository in the current directory

git init

What Happens When You Run **git init** ?

- Git creates a hidden folder called .git inside your project.
- This is where Git stores all the information it needs to track your files and history.

git status : Display the state of the working directory & staging area

git status

GIT COMMANDS



git tagging

A **tag** in Git is like a label or bookmark for a specific commit.

- `git tag <tagname>` - Create a lightweight tag
- `git tag -a <tagname> -m "message"` - Create an annotated tag
- `git tag <tagname> <commit-hash>` - Tag a specific commit
- `git tag` - List tags
- `git show <tagname>` - Show tag details

Push Tags to Remote

```
git push origin v1.0
```

Delete Tags

```
git tag -d v1.0
```

Locally

```
git push origin --delete tag v1.0
```

Remote Repository

GIT COMMANDS



git stashing

git stash lets you save your uncommitted changes and return to a clean working directory.

- `git stash` - Stash your changes
- `git stash push -m "message"` - Stash with a message
- `git stash list` - List all stashes
- `git stash branch <branchname>` - Create a branch from a stash

git stash apply

Restore your most recent stashed changes (keeps the stash in the stack)

git stash pop

Apply the latest stash **and remove it from the stack**

git stash drop

Delete a specific stash when you no longer need it

git stash clear

Delete all your stashes at once

```
git stash drop stash@{0}
Dropped stash@{0} (abc1234d5678)
```

GIT COMMANDS



git branching

In Git, a branch is like a separate workspace where you can make changes and try new ideas without affecting the main project. Think of it as a "parallel universe" for your code.

`git branch feature-branch`

Lists or create a new branch

`git checkout feature-branch`

Switches between branches or Restore files

`git switch feature-branch`

Switch Branches

`git merge feature-branch`

Combine changes from one branch into another

`git rebase main`

move your current branch on top of another branch (e.g., update your feature branch with latest main)

`git branch -d feature-branch`

Delete Branch

GIT COMMANDS



GIT BRANCHING

- `git branch` (to check branch)
- `git branch -M main` (to rename branch)
- `git checkout <-branch name->` (to navigate)
- `git checkout -b <-new branch-name->` (to create new branch)
- `git branch -d <-branch name->` (to delete branch)

GIT COMMANDS

Remote Repositories



```
git clone https://github.com/user/repo.git
```

Clones a remote repository to the local machine

```
git pull origin main
```

Fetches and merges changes from a remote repository

```
git fetch origin
```

Download changes from a repository without merging

```
git remote -v
```

List the url of the remote repositories

GIT COMMANDS



GIT IGNORE FILE

- Create .gitignore file
- Ignore unnecessary files.

```
.gitignore U X
DevLabs_2 > Profiles > .gitignore
1 *.txt
2 Assets/
```

```
node_modules
.env
*.log
```

GITHUB NEW REPOSITORY



Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1

General

Owner *

 Jaswanth-Kumar-2007

 /
introduction_devlabs is available.

Great repository names are short and memorable. How about [expert-computing-machine](#)?

Description

0 / 350 characters

2

Configuration

Choose visibility *
Choose who can see and commit to this repository

Public

Add README
READMEs can be used as longer descriptions. [About READMEs](#)

Off

Add .gitignore
.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore

Quick setup — if you've done this kind of thing before

Set up in Desktop

 or

HTTPS

SSH

Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

...or create a new repository on the command line

```
echo "# introduction_devlabs" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/Jaswanth-Kumar-2007/introduction_devlabs.git
git push -u origin main
```

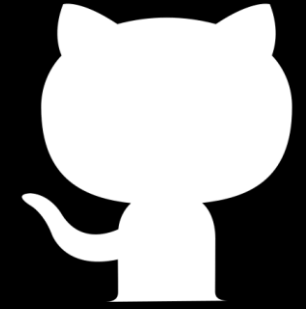
...or push an existing repository from the command line

```
git remote add origin https://github.com/Jaswanth-Kumar-2007/introduction_devlabs.git
git branch -M main
git push -u origin main
```

 **ProTip!** Use the URL for this page when adding GitHub as a remote.

 © 2026 GitHub, Inc. [Terms](#) [Privacy](#) [Security](#) [Status](#) [Community](#) [Docs](#) [Contact](#) [Manage cookies](#) [Do not share my personal information](#)

GITHUB NEW REPOSITORY



- `git init`
- `git add README.md`
- `git commit -m "first commit"`
- `git branch -M main`
- `git remote add origin https://github.com/Jaswanth-Kumar2007/introduction_devlabs.git`
- `git push -u origin main`

Add or Update the Remote Origin

To add the remote origin (first time):

Example

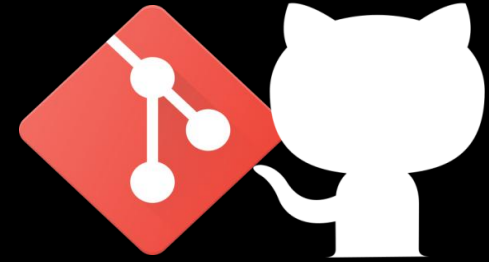
```
[user@localhost] $ git remote add origin git@github.com:your-username/your-repo.git
```

To update an existing remote to use SSH:

Example

```
[user@localhost] $ git remote set-url origin git@github.com:your-username/your-repo.git
```

GIT AND GITHUB TASK for Session



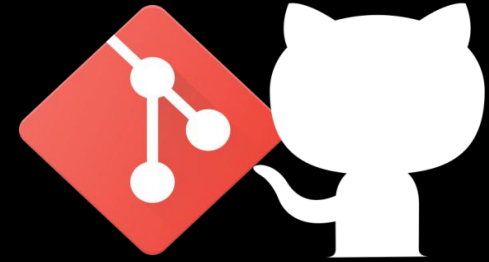
- `git clone https://github.com/Jaswanth-Kumar-2007/DevLabs_2.0.git`

```
Microsoft Windows [Version 10.0.26200.8328]  
(c) Microsoft Corporation. All rights reserved.  
  
C:\Projects\JK Projects\OpenLake\Devlabs>git clone https://github.com/Jaswanth-Kumar-2007/DevLabs_2.0.git
```

- `git branch "ID.NO."`

```
Microsoft Windows [Version 10.0.26200.8328]  
(c) Microsoft Corporation. All rights reserved.  
  
C:\Projects\JK Projects\OpenLake\Devlabs>git branch B25DS015
```

GIT AND GITHUB TASK for Session



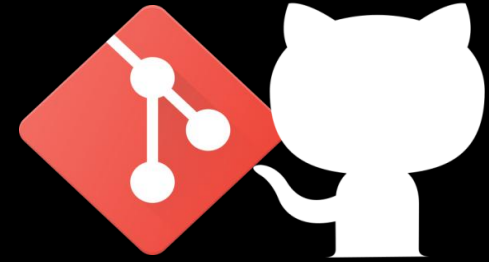
- git checkout “ID.NO.”

```
Microsoft Windows [Version 10.0.26200.8328]  
(c) Microsoft Corporation. All rights reserved.  
  
C:\Projects\JK Projects\OpenLake\Devlabs>git checkout B25DS015
```

- Create a txt File in the Profile Folder

```
B25DS015.txt U X  
DevLabs_2 > Profiles > B25DS015.txt  
1 Name : Jaswanth Kumar  
2 ID No : B25DS015  
3 Branch : DSAI  
4 Skills : HTML ,CSS , JavaScript , React
```

GIT AND GITHUB TASK for Session



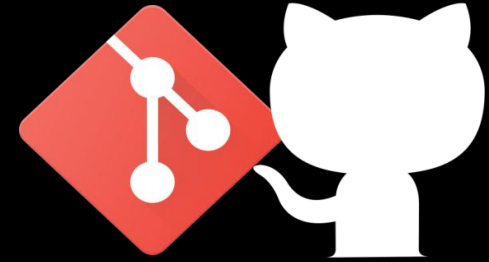
- `git add .`

```
Microsoft Windows [Version 10.0.26200.8328]  
(c) Microsoft Corporation. All rights reserved.  
  
C:\Projects\JK Projects\OpenLake\Devlabs>git add .
```

- `git commit -m {What Change you Done}`

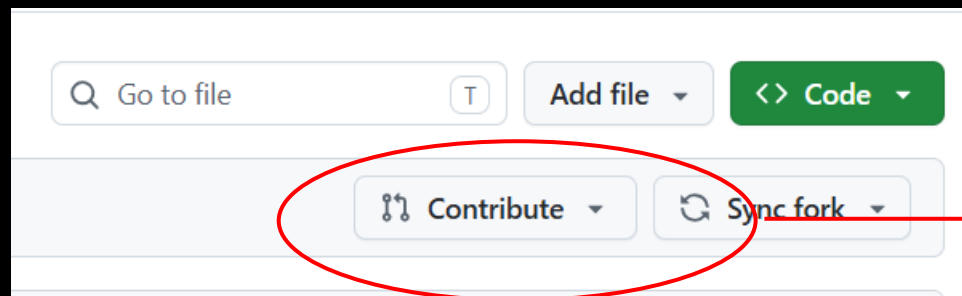
```
Microsoft Windows [Version 10.0.26200.8328]  
(c) Microsoft Corporation. All rights reserved.  
  
C:\Projects\JK Projects\OpenLake\Devlabs>git commit -m "Added Profile Txt"
```


GIT AND GITHUB TASK for Session



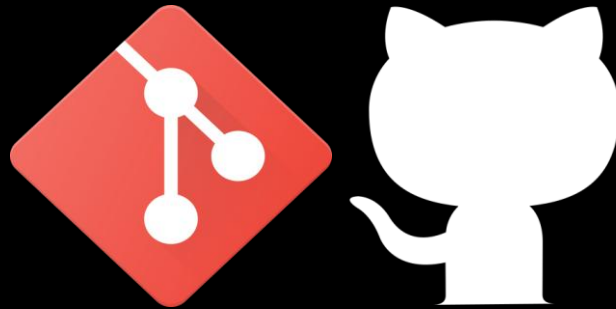
- `git push -u origin "ID.NO."`

```
Microsoft Windows [Version 10.0.26200.8328]  
(c) Microsoft Corporation. All rights reserved.  
  
C:\Projects\JK Projects\OpenLake\Devlabs>git push -u origin B25DS015
```



Push Changes to the
Repo

THANK YOU



Again meet in next Session 😊